

Base Paper

“Chain-of-Thought Prompting Elicits Reasoning in Large Language Models” by Wei, Wang, Schuurmans, Bosma, Ichter, Xia, Chi, Le & Zhou. ([arXiv](#))

The paper shows that by including intermediate reasoning steps (“chain of thought”) in prompts, large LMs drastically improve on multi-step reasoning (arithmetic, commonsense, symbolic) tasks. ([OpenReview](#))

Project Title

Math & Analytics Tutor with Step-by-Step Reasoning

Objective

Build a polished internal or customer-facing tool where a user (student, analyst, junior dev) enters a math/analytics question (e.g., “Given sales data for Q1 and Q2, compute growth rate and forecast Q3”, or “Solve the integral $\int x^2 e^x dx$ and show your work”). The system uses a large language model that is prompted with chain-of-thought style exemplars to produce not just the answer but the detailed reasoning steps. The UI shows each reasoning step clearly (“Step 1: Identify knowns”, “Step 2: Apply formula”, etc). The result should appear as a pro educational/advisory tool though the team may not be model-research experts.

Tech Stack

Backend

- Python 3.10+ with FastAPI (or Flask) to serve an endpoint `/ask` that takes question → returns reasoning steps + final answer.
- Model: Use an LLM (open-source or API; e.g., LLaMA, GPT-style) configured with chain-of-thought prompting.
- Prompt engineering: Create a prompt template with few-shot exemplars: question → reasoning chain → answer. Use that template for all questions.
- Optional: Retrieval layer (vector store) to fetch relevant formulas/data from internal knowledge base to feed into prompt.

- Storage: PostgreSQL/SQLite for logging questions, reasoning steps, answers, user_id, timestamp.
 - Frontend: React (or Vue) UI: input area for question, “Solve” button, display of reasoning steps in collapsible view, final answer. Option to download PDF of steps + answer.
 - Containerisation: Docker image for backend. Since model may be hosted via API, GPU not mandatory; if self-hosted use GPU.
 - Deployment: Cloud or on-prem depending on model; integrate with SSO if internal.
 - Logging/Monitoring: track number of questions, average latency, user feedback (“Correct?” yes/no).
 - UX: Provide example questions, prompt “Enter your problem (math / analytics)”. Show reasoning chain clearly, allow user to copy/download.
-

Dataset Information & Source

- The chain-of-thought paper did not rely on a new large labelled “dataset” per se; it uses reasoning benchmarks and few-shot exemplars. ([arXiv](#))
- For your system:
 - Create a **set of few-shot exemplar problems** (10-20) relevant to your domain (math/analytics) with detailed reasoning chains. These become part of your prompt template.
 - Use your internal data (analytics reports, typical math problems) as input domain.
 - For user questions, you don’t need to build new dataset; you rely on dynamic user input.
- External sources for problem sets you may refer to:
 - GSM8K (math word problems) example benchmark in the paper. ([arXiv](#))
 - Tutorials on chain-of-thought prompting and prompt engineering: [PromptingGuide – CoT](#) ([Papers with Code](#))

- Data flow: User question → apply prompt template with exemplars → feed into model → model outputs reasoning steps + answer → return to UI + store in DB.
-

Team Allocation & Timeline

- Team size: ~3 developers (1 prompt/model lead, 1 backend engineer, 1 frontend/devops)
 - Sprint: 8 weeks
 - Weeks 1-2: Define user personas (student/analyst), collect and craft exemplar problems + reasoning chains.
 - Weeks 3-4: Setup backend environment; build API endpoint and integrate with chosen LLM; test with exemplar prompts.
 - Week 5: Build frontend UI (question input, “Solve” button, reasoning steps display), integrate with backend.
 - Week 6: Logging and metrics, error-handling, user feedback UI.
 - Week 7: Deploy prototype into test environment, run pilot with sample users (10-20 questions), evaluate correctness and UI usability.
 - Week 8: Documentation (README + architecture diagram), UI/UX polish, final bug-fixes, demo video.
-

Deliverables & Professional-Look Checklist

- Repo with backend + frontend code; README explaining setup, model config, prompt template.
- Architecture diagram: UI → question input → backend → model call (with chain-of-thought prompt) → response → UI + logs.
- Demo: user enters question “Compute correlation coefficient given X and Y” or “Solve integral”, system shows reasoning steps and answer.

- UI: clean layout, reasoning steps clearly numbered, ability to collapse/expand each step, final answer bolded, export to PDF/Word.
 - Metrics: average latency, number of questions processed, feedback score (correct or incorrect) logged.
 - Testing: use a variety of question types (simple math, multi-step analytics, domain-specific metric computation) to verify reasoning chain is correct and interpretable.
 - Documentation: explain why chain-of-thought prompting is used, how the prompt template is structured, known limitations (model may still hallucinate reasoning).
-

Risks & Mitigations

- Risk: Model may output incorrect reasoning or jump to wrong conclusion → Mitigation: Show full reasoning chain; provide “flag answer” option; require human review for high-stakes questions.
 - Risk: Latency or cost of model API too high → Mitigation: select smaller model variant; limit max tokens; cache frequent queries/answers.
 - Risk: Exemplar prompts may not cover domain specific problems → Mitigation: periodically update exemplar library; allow user to provide context or upload data.
 - Risk: User misunderstanding of reasoning chain or steps too technical → Mitigation: provide tooltips for steps, allow “explain this step” feature, keep reasoning chain concise and clear.
-

References & Links

- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q., Zhou, D. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” arXiv: <https://arxiv.org/abs/2201.11903> ([arXiv](#))
- PromptingGuide: “Chain-of-Thought prompting” technique summary: <https://www.promptingguide.ai/techniques/cot> ([Papers with Code](#))

- Google Research blog: “Language Models Perform Reasoning via Chain of Thought”
([Google Research](#))